Practical 4

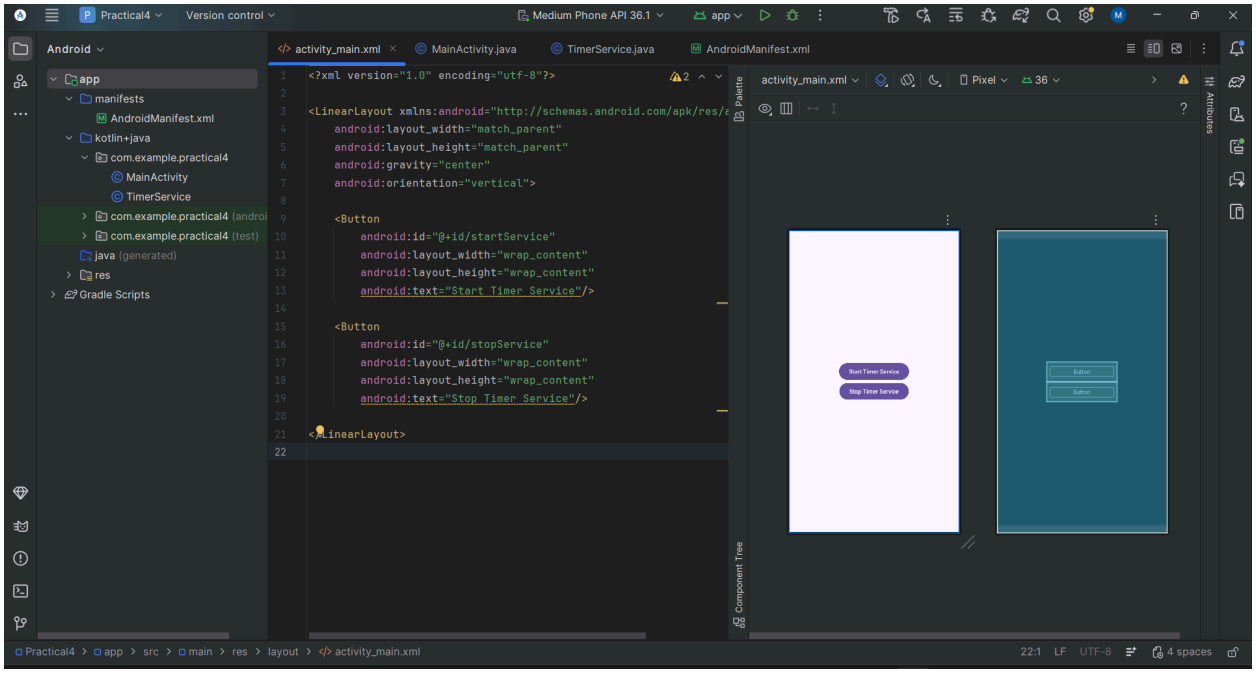
Aim: Implementing Services in Android. Develop a service that runs in the background and performs a simple task. Observe the life cycle of the service and its interaction with the main application.

Theory: This practical demonstrates Android Services, which run in the background without user interaction. It shows how a service performs a task, such as counting time using a separate thread. Service lifecycle methods such as onCreate(), onStartCommand(), and onDestroy() are used to manage execution.

Codes:

activity_main.xml - Creates UI with two buttons:

- Start Service → Starts timer
- Stop Service → Stops timer



AndroidManifest.xml - Registers the service so the system can recognise and run it.

activity_main.xml | MainActivity.java | TimerService.java | AndroidManifest.xml

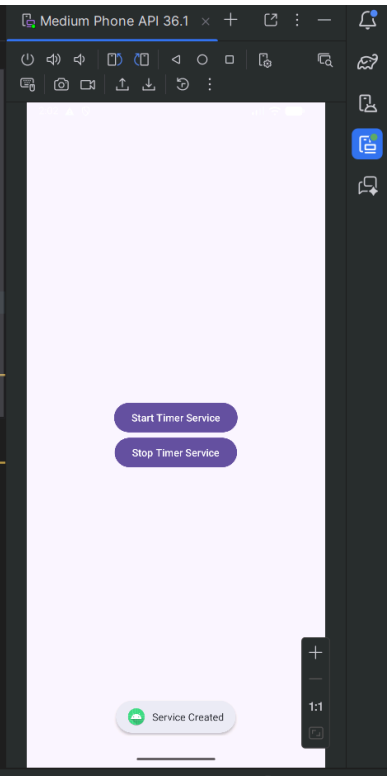
```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/Theme.Practical4">

        <service
            android:name=".TimerService"
            android:enabled="true"
            android:exported="false" />

        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>

    </application>

</manifest>
```

Output:

TimerService.java - This class defines a background service that counts seconds continuously. It runs on a separate thread and displays the timer value using Toast messages every second.

activity_main.xml | MainActivity.java | TimerService.java | AndroidManifest.xml

```java
package com.example.practical4;
import android.app.Service;
import android.content.Intent;
import android.os.IBinder;
import android.widget.Toast;

public class TimerService extends Service {
    2 usages
    int seconds = 0;
    2 usages
    boolean running = true;

    @Override
    public void onCreate() {
        super.onCreate();
        Toast.makeText( context: this,  text: "Service Created", Toast.LENGTH_SHORT).show();
    }

    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {

        new Thread(new Runnable() {
            @Override
            public void run() {

                while(running)
                {
                    try {
                        Thread.sleep( millis: 1000);
                        seconds++;

                        Toast.makeText(getApplicationContext(),
```

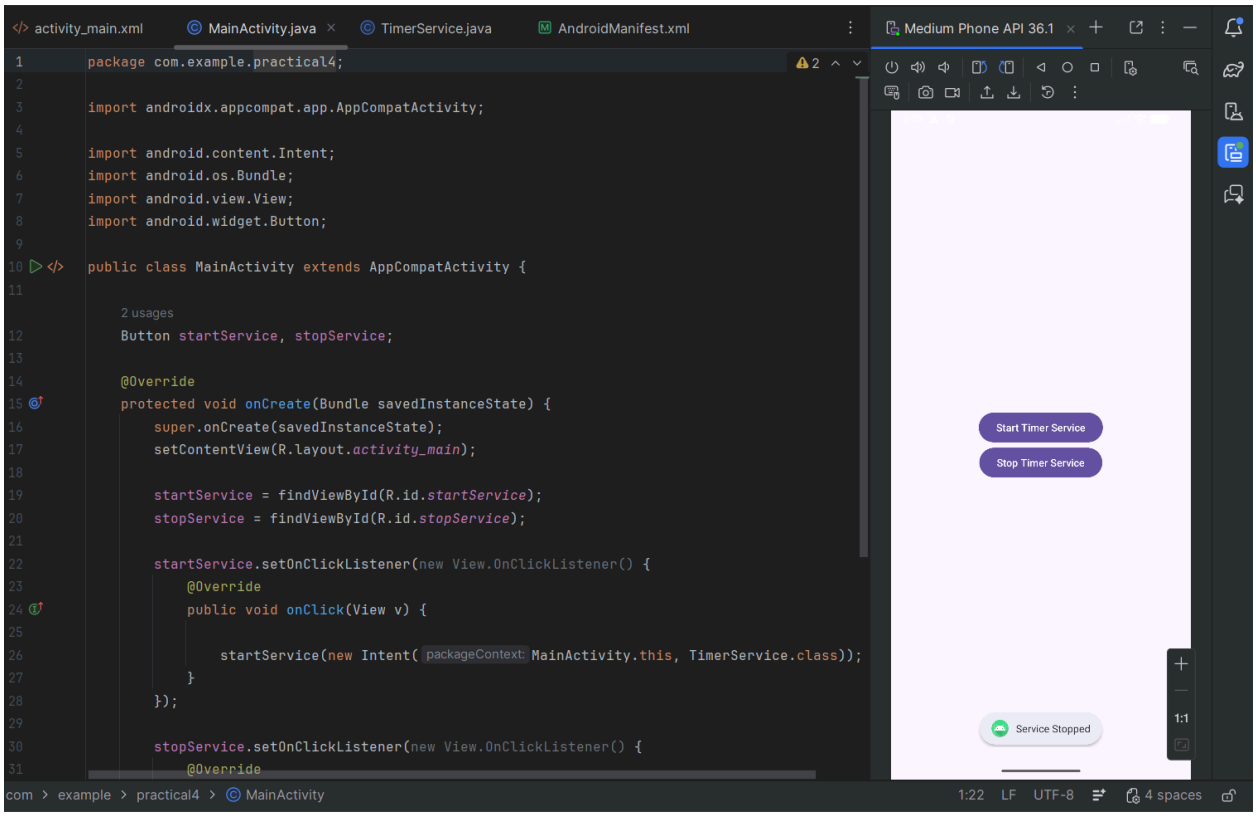


com > example > practical4 > TimerService 10:1 LF UTF-8 4 spaces

Code Explanation

- onCreate() → Called when service is created
- onStartCommand() → Starts background thread for timer
- Thread.sleep(1000) → Delays execution by 1 second
- seconds++ → Increments timer count
- Toast.makeText() → Displays timer value
- onDestroy() → Stops service and thread
- onBind() → Returns null (not a bound service)

MainActivity.java - This activity provides buttons to start and stop the service. It uses Intent to control the service execution.



Code Explanation

- startService() → Starts TimerService
- stopService() → Stops TimerService
- setOnClickListener() → Handles button clicks

Conclusion

The Android Service was successfully implemented. The service ran in the background, performed the timer task, and demonstrated its lifecycle methods effectively.